Q15. Analogical Prompting Write a prompt that asks the model to first recall 1–2 similar problems it has solved before, then use that reasoning to solve a new problem — for example, debugging a new piece of code by referencing a similar bug pattern.

Aim

To design an analogical prompt that enables the AI model to recall similar previously solved problems and apply that reasoning to solve a new problem effectively.

Python Code:

```
from openai import OpenAI

# Paste your OpenAI API Key here
client = OpenAI(
    api_key="sk-proj-
fXs9TwmdiZQU9td92kDIe9wACeeq8S0wsiGwQRfrbEoJq4VGPLU1Np74KCiIODlH8_C
oBWZp8nT3BlbkFJBjvFWrf8sOhvLZ7zO3YVr2MDh2NeIM42jru5-
z2nctrTGHuEgdMAYbFx2lo34b4CY-q8ffMBkA"
)

analogical_prompt = """
You are an expert software engineer.

First, recall one or two similar debugging problems that involve a program
crashing because of a missing module or syntax error.

Briefly explain how those problems were solved.

Then use the same reasoning to solve this new problem:

Problem:
A Python program raises the error:
ModuleNotFoundError: No module named 'numpy'

Provide the solution step by step and explain why it works.
"""

response = client.responses.create(
    model="gpt-5.4-mini",
    input=analogical_prompt
)

print("===== ANALOGICAL PROMPT =====")
```

print(analogical_prompt)

print("\n===== OUTPUT =====")
print(response.output_text)

Output:

===== ANALOGICAL PROMPT =====

You are an expert software engineer.

First, recall one or two similar debugging problems that involve a program
crashing because of a missing module or syntax error.

Briefly explain how those problems were solved.

Then use the same reasoning to solve this new problem:

Problem:
A Python program raises the error:
ModuleNotFoundError: No module named 'numpy'

Provide the solution step by step and explain why it works.

===== OUTPUT =====

A couple of similar debugging cases:

1. **`ModuleNotFoundError: No module named 'requests'`**
   This usually happened because the package was not installed in the active Python environment.
   Fix: run `pip install requests` (or `python -m pip install requests`) in the same environment the program uses.

2. **`SyntaxError: invalid syntax`**
   This often came from a typo, a missing colon, mismatched parentheses, or using Python 2 syntax in Python 3.
   Fix: inspect the line number in the traceback, correct the typo or syntax, and rerun the program.

Applying the same reasoning to the new problem

Error
`ModuleNotFoundError: No module named 'numpy'`

This means Python tried to import `numpy`, but it is **not installed** in the environment currently running the program.

Step-by-step solution

1) Check which Python environment is running the program
The program may be using a different interpreter than the one you expect, especially if you use:
- virtual environments
- Conda
- multiple Python versions
- IDEs like VS Code or PyCharm

You can check with:

```bash
python --version
python -m pip --version
```

If needed, use:

```bash
python -c "import sys; print(sys.executable)"
```

This shows the exact Python executable being used.

2) Install NumPy in that same environment
Use:

```bash
python -m pip install numpy
```

Why this works:
- `python -m pip` ensures `pip` is tied to the same Python interpreter that runs your script.
- This avoids installing NumPy into the wrong environment.

If you are using Python 3 and `python` points to Python 2 or another interpreter, use:

```bash
python3 -m pip install numpy
```

3) Verify the installation
Run:

```bash
python -c "import numpy; print(numpy.__version__)"
```

If this prints a version number, NumPy is installed correctly.

4) Re-run your program
Now the import should succeed:

```python
import numpy as np
```

Why this works
Python imports modules from the packages installed in the current environment.
The error occurs because NumPy is missing there. Installing it adds the package to Python’s searchable module path, so `import numpy` can find and load it.

If it still fails
If the error continues after installation, the most likely cause is that the program is running in a **different environment** than the one where you installed NumPy. In that case:
- activate the correct virtual environment
- select the correct interpreter in your IDE
- reinstall with `python -m pip install numpy`

If you want, I can also show the exact commands for **venv**, **Conda**, or **VS Code**..

Result

The analogical prompt successfully guided the AI model to reference similar bug patterns before solving the new debugging task, resulting in a more accurate and logical solution